Payments platform on-call runbook

Runbook PAY-OPS-004, revision 4.2 · Owner: Payments Reliability · Reviewed 30 September 2026

Bridge Join `#inc-payments` and the standing video bridge before anything else.

Dashboards grafana.example/d/payments-overview

Status page Customer updates go out through the support duty manager only.

1 Scope and on-call duties

This runbook covers the card, wallet, and bank-transfer paths of the payments platform in both production regions. It tells the on-call engineer **what to check, in what order, and when to escalate**. It does not replace judgement: if a step makes things worse, stop and escalate using section 5.

- Acknowledge every page within five minutes, day or night.
- Keep a laptop with a working VPN and hardware key within reach.
- Hand over at 09:00 local time with a written summary:
  - open incidents and their current severity;
  - changes frozen, rolled back, or still in flight.

2 Service topology

Every payment enters through the edge, is authorised by the payments API, and is written to the ledger before any acquirer is told to capture funds. Figure 1 shows the tiers you will meet during an incident.

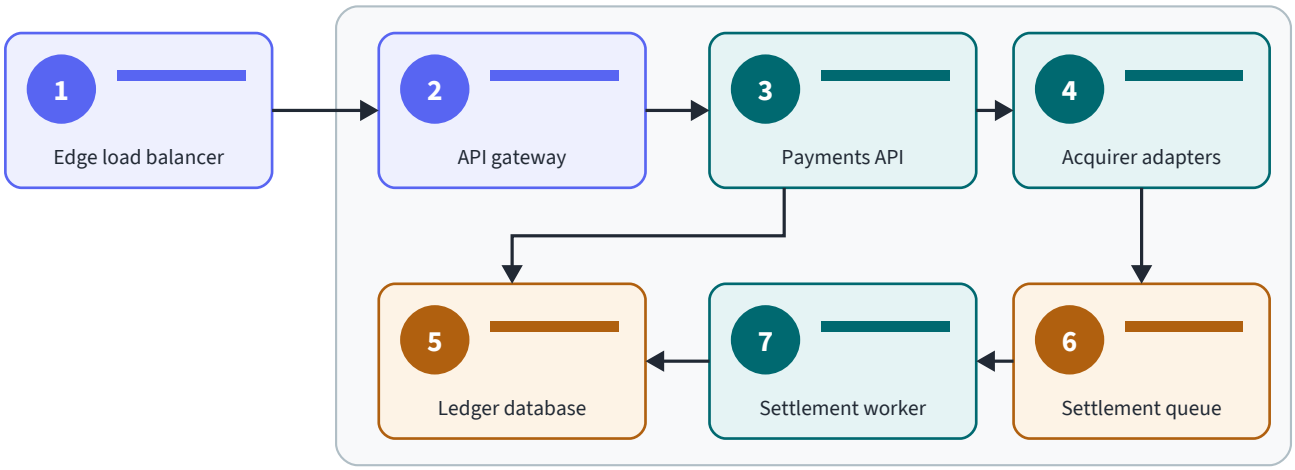


Figure 1. Request path through the production zone (shaded)

1. **Edge load balancer** terminates TLS and applies rate limits.
2. **API gateway** authenticates merchants and routes by region.
3. **Payments API** validates, deduplicates, and authorises each payment.
4. **Acquirer adapters** speak each bank's protocol and own the retries.
5. **Ledger database** is the system of record; a primary with two replicas.
6. **Settlement queue** buffers captures for the nightly settlement run.
7. **Settlement worker** drains the queue and reconciles against the ledger.

3 Severity levels

Declare the severity as soon as the impact is understood, and change it as soon as the impact changes. Response targets run from the moment the page fires, not from acknowledgement.

Table 1. Severity levels and response targets

Level	Meaning	Response	Example
SEV1	Customers cannot pay, or money is at risk.	Page now; bridge in 5 min; updates every 15 min.	*Card authorisations failing in every region.*
SEV2	A major feature is degraded for many customers.	Page now; updates every 30 min.	*Refunds delayed by more than an hour.*
SEV3	Minor impact with a workaround.	Next business day.	*One acquirer returns slow responses.*
SEV4	No customer impact yet.	Ticket in the team queue.	*A certificate expires in 21 days.*

4 Incident response procedure

Work through the steps in order. Record every action, with the time, in the incident channel as you take it: the timeline is written from those messages.

4.1 Triage

1. Acknowledge the page and open the incident with `/incident open payments`.
2. Check the overview dashboard for the three golden signals: authorisation rate, p99 latency, and error rate.
3. Decide whether the fault is ours or upstream:
   a. if one acquirer is failing, disable it and let routing fail over;
   b. if every acquirer is failing, suspect the payments API or the ledger;
   c. if only one region is failing, drain it at the edge.
4. Declare the severity using Table 1 and page the next level if the target is at risk.

```
$ kubectl -n payments get pods -l tier=api -o wide
$ kubectl -n payments logs deploy/payments-api --since=15m
$ payctl acquirer status --region ap-southeast-2
```

4.2 Rolling back a release

Most incidents start within an hour of a deploy. If the timing fits, **roll back first and investigate afterwards**.

1. Freeze deploys with `payctl freeze --reason INC`.
2. Find the last good revision with `kubectl rollout history`.
3. Roll back with `kubectl rollout undo --to-revision=N` and watch the pods restart.
4. Confirm the authorisation rate has recovered for ten minutes before you lift the freeze.

> **Warning** Never roll back across a ledger schema migration.
>
> Check `payctl migrations --pending` first. If a migration ran, escalate to the database specialist.

4.3 Database failover

Fail the ledger over only when the primary is unreachable for more than two minutes and the incident commander agrees. Failover is automatic in most cases; these steps are for when it is not.

1. Confirm replication lag on both replicas is below one second.
2. Promote the healthier replica with `ledgerctl promote --replica`.
3. Repoint the payments API by restarting it; it resolves the primary at start-up.
4. Fence the old primary so it cannot accept writes if it returns.

```
$ ledgerctl replicas --lag
$ ledgerctl promote --replica ledger-2 --confirm
$ kubectl -n payments rollout restart deploy/payments-api
```

> **Note** A failover loses no committed payment: writes are synchronous to one replica.

5 Escalation

Escalate on the clock, not on a feeling. If an incident is still open at the time shown, page the next role even if you believe a fix is close.

Table 2. Escalation path for SEV1 and SEV2 incidents

After	Escalate to	Who	Channel
0 min	**Primary on-call**	Payments rota (PagerDuty)	`#inc-payments`
10 min	**Secondary on-call**	Payments rota, backup	`page: pay-secondary`
20 min	**Incident commander**	Duty engineering manager	`page: ic-duty`
30 min	**Database specialist**	Data platform rota	`page: dba-oncall`
45 min	**Head of Payments**	Priya Raman	`+61 3 9000 4417`
60 min	**Customer comms lead**	Support duty manager	`#status-updates`

Escalate earlier whenever you are unsure. Nobody is ever blamed for paging.

6 After the incident

Close the incident only when the metrics have been healthy for thirty minutes. Within two business days:

- write the timeline from the incident channel;
- hold a blameless review with everyone who was paged;
- file follow-up actions against the `PAY-REL` board with an owner and a date;
- update this runbook if any step was wrong, missing, or slow.

Suggest changes to this runbook in `#payments-reliability`.

Appendix A. Command reference

Every command below is read-only unless it is marked as a write. Run writes only while the incident is open and announced in the channel.

Table 3. Commands used in this runbook

Command	Purpose	Writes
`/incident open payments`	Open an incident channel and start the timeline.	Yes
`kubectl get pods -l tier=api`	List API pods with their node and restart count.	No
`kubectl logs deploy/payments-api`	Stream recent API logs.	No
`payctl acquirer status`	Show authorisation rate per acquirer.	No
`payctl acquirer disable NAME`	Stop routing to a failing acquirer.	Yes
`payctl freeze --reason INC`	Block all deploys to the payments namespace.	Yes
`payctl migrations --pending`	List ledger migrations applied since the last release.	No
`kubectl rollout undo`	Return a deployment to an earlier revision.	Yes
`ledgerctl replicas --lag`	Show replication lag for each ledger replica.	No
`ledgerctl promote --replica`	Promote a replica to ledger primary.	Yes

Appendix B. Procedure drills

Each procedure is rehearsed in the staging region on the schedule below. A procedure with no drill date has not been rehearsed since it was added; run it with its owner before you rely on it.

Table 4. Drill schedule

Procedure	Owner	Last drill	Next drill
4.1 Triage	Payments Reliability	12 Aug 2026	11 Nov 2026
4.2 Rolling back a release	Release engineering	2 Sep 2026	2 Dec 2026
4.3 Database failover	Data platform		14 Oct 2026
5 Escalation	Duty engineering managers	19 Aug 2026	18 Nov 2026